

Solving Combinatorial Optimization Problems

Greedy, dynamic programming, approximation — and the Ising/QUBO form

Lecture 3

September 9, 2026

Outline

- In Lecture 2 we learned to *model* a combinatorial optimization problem
 - an instance x , a feasible set $S(x)$, a cost function $c(x, y)$
 - TSP, Knapsack and Max-Cut all fit the same three-part template
- But a model only says *what* a solution is — not *how* to find one
- Today: three practical strategies for actually finding y^*
 - **Greedy** — fast, and sometimes wrong
 - **Dynamic programming** — exact, and sometimes too expensive
 - **Approximation** — fast, with a promise attached
- Then we meet the **Ising model** and its **QUBO** form — the shape every quantum optimizer in this course expects its input to be in

We will take a detour!

The following slides are NOT part of any assessment

Disclaimer: these slides were generated using AI for purpose of illustrating algorithm strategies

Why not just try everything?

- The feasible set $S(x)$ is finite, so brute force always works
- TSP on n cities: $(n - 1)!/2$ distinct tours | Knapsack on n items: 2^n subsets

n	TSP tours $(n - 1)!/2$	Knapsack subsets 2^n	enumerate tours at $10^9/s$
10	1.8×10^5	1.0×10^3	instant
15	4.4×10^{10}	3.3×10^4	44 seconds
20	6.1×10^{16}	1.0×10^6	about 2 years
25	3.1×10^{23}	3.4×10^7	about 10^7 years
50	3.0×10^{62}	1.1×10^{15}	about 10^{46} years

Takeaway: Brute force is correct and useless. Everything else in this course is an attempt to avoid it.

Three strategies

Greedy, dynamic programming, approximation

Greedy: the idea

- At every step, take the choice that looks best *right now*
- Never reconsider a choice already made
- No search, no backtracking — so it is cheap, typically $O(n^2)$ or better
- It is genuinely optimal for some problems
 - Kruskal's algorithm for minimum spanning tree
 - *Fractional* knapsack (you may take a fraction of an item)
 - Huffman coding, interval scheduling by earliest finish time
- And it can be arbitrarily bad for others

The catch: A cheap choice made early can force an expensive choice later — and greedy has no way to see that coming.

Greedy for TSP: nearest neighbour

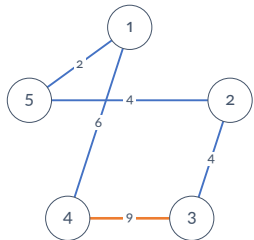
- Start at city 1 and mark it visited
- Repeatedly travel to the *nearest unvisited* city
- When every city is visited, return to city 1
- An instance with $V = \{1, 2, 3, 4, 5\}$ and symmetric w
- $S(x)$ contains $4!/2 = 12$ distinct tours
- Note the single expensive edge:
 $w(3, 4) = 9$

	1	2	3	4	5
1	—	5	4	6	2
2	5	—	4	5	4
3	4	4	—	9	6
4	6	5	9	—	5
5	2	4	6	5	—

Running nearest neighbour

at	unvisited (cost)				take
1	2:5	3:4	4:6	5:2	→ 5 (2)
5	2:4	3:6	4:5		→ 2 (4)
2	3:4	4:5			→ 3 (4)
3	4:9				→ 4 (9)
4	— (all visited)				→ 1 (6)

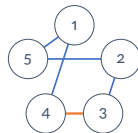
- Tour: $1 \rightarrow 5 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1$
- Cost: $2 + 4 + 4 + 9 + 6 = \mathbf{25}$



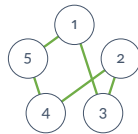
What went wrong

- Nearest neighbour: $1 \rightarrow 5 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1$, cost **25**
- Optimal tour: $1 \rightarrow 3 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 1$, cost **20**
 - $4 + 4 + 5 + 5 + 2 = 20$, checked against all 12 tours
- Greedy is 25% worse than optimal on a five-city instance
- **Why:** greedy spent all the cheap edges early
- Edge (3, 4) costs 9 — the most expensive in the graph
- Greedy declined it at every step... and was *forced* into it at the end, when city 4 was the only city left

nearest neighbour: 25



optimal: 20



So when *does* greedy work?

- Greedy is exactly optimal when the problem has the right structure
 - Minimum spanning tree (Kruskal): always optimal
- Treat greedy as a *heuristic*: a good starting point, never a proof

Takeaway: Greedy gives you an answer fast. It never tells you how far from optimal that answer is.

Dynamic programming: the idea

- Two properties make a problem a candidate:
 - **Optimal substructure:** an optimal solution is built out of optimal solutions to smaller versions of the same problem
 - **Overlapping subproblems:** those smaller versions repeat, over and over
- So: solve each subproblem *once*, write the answer in a table, and look it up every later time it is needed
- Brute force re-derives the same subproblem exponentially many times; DP pays for it once
- Unlike greedy, DP does not commit early — it keeps *both* options (“take it” and “leave it”) and compares them with full information

DP for 0/1 Knapsack: the recurrence

- Let $K[i][c]$ = the best value achievable using only items $1, \dots, i$ with capacity exactly c available
- Item i is either left out, or taken — and we compare the two:

$$K[i][c] = \begin{cases} K[i-1][c], & \text{if } w_i > c \quad (\text{it does not fit}) \\ \max(\underbrace{K[i-1][c]}_{\text{leave item } i}, \underbrace{v_i + K[i-1][c - w_i]}_{\text{take item } i}), & \text{otherwise} \end{cases}$$

- Base row: $K[0][c] = 0$ for every c — no items, no value
- The answer we want is $K[n][W]$

Filling the table

- 3 items, capacity $W = 4$:
 - item 1: $w_1 = 2, v_1 = 3$
 - item 2: $w_2 = 3, v_2 = 4$
 - item 3: $w_3 = 4, v_3 = 5$
- Fill row by row, left to right
- Example, $K[2][4]$: item 2 fits, so compare
 - leave it: $K[1][4] = 3$
 - take it: $4 + K[1][1] = 4 + 0 = 4$
 - so $K[2][4] = 4$

$K[i][c]$	$c=0$	1	2	3	4
$i = 0$ (none)	0	0	0	0	0
$i = 1$ (2, 3)	0	0	3	3	3
$i = 2$ (3, 4)	0	0	3	4	4
$i = 3$ (4, 5)	0	0	3	4	5

Optimal value = $K[3][4] = 5$

Reading the solution back — and the price

- The table gives the optimal *value*; walk back up it to recover y^*
 - $K[3][4] = 5 \neq K[2][4] = 4$, so item 3 *was* taken; capacity drops to $4 - 4 = 0$
 - $K[2][0] = K[1][0] = K[0][0] = 0$, so items 2 and 1 were not taken
 - $y^* = (0, 0, 1)$, weight 4, value 5 — and greedy gave 3
- **Cost:** the table has $n \times W$ cells, each filled in $O(1)$, so $O(nW)$
- That looks polynomial — but it is not
 - W is a *value* written in the input, not the input's *size*
 - writing $W = 10^9$ takes 30 bits; the table would need 10^9 columns
 - this is called **pseudo-polynomial** time

The catch: DP is exact, and its cost is predictable — but “predictable” can still mean “impossible”.

Approximation: the idea

- Stop insisting on the optimum. Insist on a *bound* instead
- An algorithm is a ρ -**approximation** for a minimization problem if, on *every* instance,

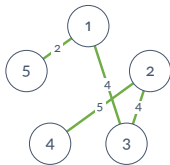
$$c(x, y_{\text{ALG}}) \leq \rho \cdot c(x, y^*), \quad \rho \geq 1$$

- $\rho = 2$ means: never more than twice the optimal cost, no matter the input
- The subtle and useful part: we get this promise *without ever computing* $c(x, y^*)$
 - the trick is to compare the algorithm against a quantity we *can* compute and that we know sits below the optimum — a *lower bound*
- Nearest neighbour has no constant ρ — for general TSP it can be made arbitrarily bad. We need extra structure

Minimum spanning tree

- A **spanning tree** of a connected graph on n vertices is a set of edges that touches every vertex, keeps it connected, and contains no cycle — exactly $n - 1$ edges
- The **minimum** spanning tree T is the spanning tree of least total weight $w(T)$
- **Kruskal's algorithm** builds it greedily — and here greedy is *exactly* optimal: sort the edges by weight, take the next one unless it closes a cycle, stop after $n - 1$

edge	w	
(1, 5)	2	take
(1, 3)	4	take
(2, 3)	4	take
(2, 5)	4	skip — cycle
(1, 2)	5	skip — cycle
(2, 4)	5	take — 4th, stop



$$T = \{(1, 5), (1, 3), (2, 3), (2, 4)\}, w(T) = 15$$

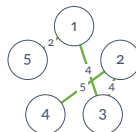
Metric TSP, and a free lower bound

- Assume the **triangle inequality**:

$$w(i, k) \leq w(i, j) + w(j, k)$$

- “Going directly is never worse than detouring through a third city” — true of road, air and straight-line distances
- Key fact:** $w(\text{MST}) \leq c(x, \pi^*)$
 - delete one edge from an optimal tour: what remains is a path through all n cities, which is a spanning tree — so the *minimum* spanning tree can only be cheaper
- The MST is computable in $O(n^2 \log n)$, and it sits below the optimum for free

MST of our instance:
 $\{(1, 5), (1, 3), (2, 3), (2, 4)\}$
weight = $2 + 4 + 4 + 5 = 15$



The 2-approximation for metric TSP

1. Build a minimum spanning tree T
2. Walk T depth-first from any root, recording each city the *first* time you reach it
3. Visit the cities in that order and return to the start

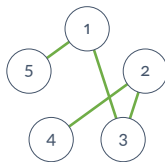
- **Why it is within a factor of 2:**

- a full depth-first walk of T crosses every tree edge exactly twice, so it costs $2 w(T)$
 - that walk revisits cities; “shortcutting” past an already-visited city replaces two edges by one, which the triangle inequality says cannot cost more
 - therefore $c(\text{tour}) \leq 2 w(T) \leq 2 c(x, \pi^*)$
- Both inequalities are things we can *prove*; neither requires knowing π^*

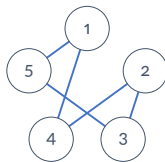
Running it on our instance

- MST edges $(1, 5), (1, 3), (3, 2), (2, 4)$, weight $w(T) = 15$
- Guarantee: the tour we produce costs at most $2 \times 15 = \mathbf{30}$
- Depth-first from city 1, visiting 5 before 3:
 - order 1, 5, 3, 2, 4
 - tour $1 \rightarrow 5 \rightarrow 3 \rightarrow 2 \rightarrow 4 \rightarrow 1$
 - cost $2 + 6 + 4 + 5 + 6 = \mathbf{23}$
- Depth-first visiting 3 before 5 instead:
 - order 1, 3, 2, 4, 5 $\Rightarrow$ cost $\mathbf{20}$ — the optimum, by luck
- Both obey the bound. The guarantee is a worst case, not a prediction

MST, weight 15



shortcut tour, cost 23



Scorecard on the same instance

method	tour	cost	guarantee	work
brute force	1 3 2 4 5 1	20	exact	$(n-1)!/2$
nearest neighbour	1 5 2 3 4 1	25	<i>none</i>	$O(n^2)$
MST shortcutting	1 5 3 2 4 1	23	$\leq 2 \cdot \text{OPT} = 30$	$O(n^2 \log n)$

- On this instance the approximation happened to beat greedy — that is not the point
- The point is that only one row of this table comes with a *promise*
- Better guarantees exist: Christofides' algorithm gives $\rho = 3/2$ for metric TSP

Takeaway: Greedy: no promise. DP: exact but expensive. Approximation: a bounded gap you can quote before you run it.

The Ising model and QUBO

One cost function to rule them all

Back to track!
The following slides are part of the assessment

Ising Model

- Instance:

$$x = (G = (V, E), J_{ij}, h_i)$$

- $G = (V, E)$ is a graph with vertices = spins and edges = couplings
 - Spin variable $s_i \in \{+1, -1\}, i \in V$
 - J_{ij} is the interaction strength between i and j
 - h_i is the external magnetic field on i
- Feasible Solutions:
- All possible spin assignments $y = (s_1, s_2, \dots, s_n), s_i \in \{+1, -1\}$
 - there are 2^n of them, and no constraints — every assignment is feasible
- Goal: find the spin configuration y that minimizes the energy $H(s)$ (the *ground state*)

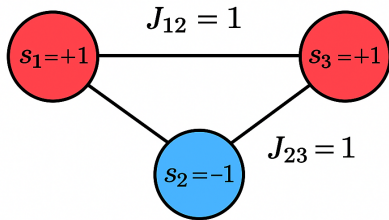
Ising Model

- Cost function:

$$c(x, y) = H(s) = - \sum_{(i,j) \in E} J_{ij} s_i s_j - \sum_{i \in V} h_i s_i$$

- First term: interaction energy (neighbour alignment vs. conflict)
- Second term: contribution from the external field
- $J_{ij} > 0$ (ferromagnetic) rewards $s_i = s_j$; $J_{ij} < 0$ (antiferromagnetic) rewards $s_i \neq s_j$

Ising Model

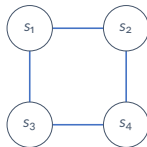


Ising Model Toy Example

- Four spins on a square: $s_1, s_2, s_3, s_4 \in \{-1, +1\}$
- Edges: $(1, 2), (1, 3), (2, 4), (3, 4)$
- All $J_{ij} = 1$ (ferromagnetic) and all $h_i = 0$

$$H(s) = -(s_1s_2 + s_1s_3 + s_2s_4 + s_3s_4)$$

- Every term is -1 when the two spins agree, $+1$ when they disagree
- So the energy is lowest when *all four* spins agree
 - $s = (+1, +1, +1, +1) \Rightarrow H = -4$
 - $s = (-1, -1, -1, -1) \Rightarrow H = -4$
 - two ground states — flipping every spin is a symmetry of the model
 - $s = (+1, -1, -1, -1) \Rightarrow H = 0$



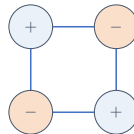
every edge has $J_{ij} = 1$

Ising Model Toy Example — antiferromagnetic

- Same four spins, same square — but now every $J_{ij} = -1$, still $h_i = 0$

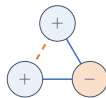
$$H(s) = - \sum_{(i,j) \in E} J_{ij} s_i s_j = + (s_1 s_2 + s_1 s_3 + s_2 s_4 + s_3 s_4)$$

- The sign flip reverses what the model rewards: each term is now -1 when the two spins *disagree*
- So the energy is lowest when neighbours *alternate*
 - $s = (+1, -1, -1, +1) \Rightarrow H = -4$
 - $s = (-1, +1, +1, -1) \Rightarrow H = -4$ (every spin flipped)
 - all four aligned $\Rightarrow H = +4$: the ferromagnetic ground state is now the *worst* state, and vice versa
- Read as Max-Cut: the two colours are the partition, and all 4 edges are cut — cut $= \frac{1}{2} (4 - (-4)) = 4$



every edge has $J_{ij} = -1$

$$s_1 = s_4 = +1, s_2 = s_3 = -1$$



a triangle cannot satisfy all three

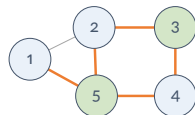
The catch: The square is an even cycle, so every bond can be satisfied at once. On an odd cycle one bond must break — that is *frustration*, and it is where hard instances come from.

Max-Cut *is* an Ising model

- Encode the partition with spins: $s_i = +1$ if $i \in S$,
 $s_i = -1$ otherwise
- Edge (i, j) is cut exactly when $s_i s_j = -1$, so

$$\text{cut}(S) = \sum_{(i,j) \in E} w_{ij} \frac{1 - s_i s_j}{2} = \frac{1}{2} \sum_{(i,j) \in E} w_{ij} - \frac{1}{2} \sum_{(i,j) \in E} w_{ij} s_i s_j$$

- The first sum is a constant. So **maximizing the cut** is the same as **minimizing** $\sum w_{ij} s_i s_j$
- That is exactly $H(s)$ with $J_{ij} = -w_{ij}$ and $h_i = 0$ — an *antiferromagnetic* Ising model
- Ground energy of this instance is -4 , and $\text{cut} = \frac{1}{2}(6 - (-4)) = 5$ — which answers the question left open at the end of Lecture 2



L2's graph, all $w_{ij} = 1$.
 $S = \{3, 5\}$ cuts 5 of the 6 edges

Quadratic Unconstrained Binary Optimization

- QUBO is a form of optimization problem
- Same idea, binary variables instead of spins
- Given n binary variables $x \in \{0, 1\}^n$ and a real symmetric $Q \in \mathbb{R}^{n \times n}$:

$$\min_{x \in \{0, 1\}^n} f_Q(x) = x^T Q x$$

- Expanded, with the diagonal carrying the linear terms:

$$f_Q(x) = \sum_{i=1}^n Q_{ii} x_i + \sum_{1 \leq i < j \leq n} Q_{ij} x_i x_j$$

- Why the diagonal is linear: $x_i \in \{0, 1\}$ means $x_i^2 = x_i$
- **Unconstrained:** there is no $S(x)$ to respect — all 2^n assignments are legal
 - constraints have to be folded into the objective as penalty terms (Lecture 4)

Ising $\leftrightarrow$ QUBO: the substitution

- QUBO uses $x_i \in \{0, 1\}$; Ising uses $s_i \in \{-1, +1\}$
- These two sets are in bijection, by a change of variable:

$$s_i = 2x_i - 1 \quad \Longleftrightarrow \quad x_i = \frac{s_i + 1}{2}$$

x_i	s_i
0	-1
1	+1

- The map is one-to-one and onto, so it carries feasible solutions to feasible solutions and preserves which one is best
- Ising and QUBO are therefore *the same problem written in different variables*

Ising $\rightarrow$ QUBO: doing the algebra

- Substitute $s_i = 2x_i - 1$ into the product term:

$$s_i s_j = (2x_i - 1)(2x_j - 1) = 4x_i x_j - 2x_i - 2x_j + 1$$

- Put that into $H(s) = -\sum_{(i,j) \in E} J_{ij} s_i s_j - \sum_{i \in V} h_i s_i$ and collect terms:

$$H(x) = \underbrace{-4 \sum_{(i,j) \in E} J_{ij} x_i x_j}_{\text{quadratic}} + \underbrace{\sum_{i \in V} \left(2 \sum_{j \in N(i)} J_{ij} - 2h_i \right) x_i}_{\text{linear}} + \underbrace{\sum_{i \in V} h_i - \sum_{(i,j) \in E} J_{ij}}_{\text{constant}}$$

- So the QUBO matrix is read straight off the Ising instance:

$$Q_{ij} = -4J_{ij} \quad \text{for } (i,j) \in E, \quad Q_{ii} = 2 \sum_{j \in N(i)} J_{ij} - 2h_i$$

- $N(i)$ is the set of neighbours of i . The constant shifts every energy equally, so it changes the energy *value* but never the *argmin*

Worked conversion: the 4-spin toy

- Instance: edges $(1, 2), (1, 3), (2, 4), (3, 4)$, all $J_{ij} = 1$, all $h_i = 0$, every vertex of degree 2
- Apply the formulas: $Q_{ij} = -4$ on each edge, $Q_{ii} = 2(1 + 1) - 0 = 4$, constant $= 0 - 4 = -4$

$$H(x) = -4(x_1x_2 + x_1x_3 + x_2x_4 + x_3x_4) + 4(x_1 + x_2 + x_3 + x_4) - 4$$

- Check it against the spin picture:
 - $x = (1, 1, 1, 1)$ ($s = \text{all} + 1$): $-16 + 16 - 4 = -4 \checkmark$
 - $x = (0, 0, 0, 0)$ ($s = \text{all} - 1$): $0 + 0 - 4 = -4 \checkmark$
 - $x = (1, 0, 0, 0)$ ($s = (+1, -1, -1, -1)$): $0 + 4 - 4 = 0 \checkmark$
- Both ground states survive the map, at the same energy -4

Where this leaves us

- Three classical strategies, and what each one buys you:
 - greedy — fast, no guarantee (nearest neighbour: 25 vs. optimal 20)
 - dynamic programming — exact, $O(nW)$ but pseudo-polynomial
 - approximation — fast, with a provable bound ($\leq 2 \cdot \text{OPT}$ for metric TSP)
- And one common target: every problem we have modelled can be pushed into Ising form, and Ising and QUBO are the same thing in different variables
- **Next (Lecture 4):**
 - why these problems are hard in the first place — P, NP, and reductions
 - how constraints get folded into an unconstrained QUBO as penalties
 - what quantum hardware actually does with an Ising Hamiltonian

■ *If Max-Cut, Knapsack and TSP can all be written as one Ising model, what exactly is the quantum computer being asked to do?*